UNIVERSIDAD EVANGÉLICA DE EL SALVADOR

Facultad de Ingeniería

Ingeniería en Desarrollo de Software y Ciencia de Datos

Programación II

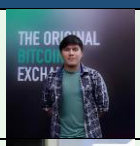
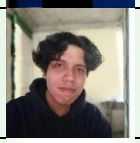
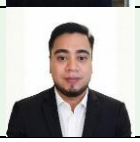
Entrega final (Avance 3)

Sistema para Administración de Asociación Comunal

Grupo 10

Facilitador: Ing. Daniel Enrique Guevara Gómez

San Salvador, El Salvador

Foto	Nombre completo	CIF
	Marcos Armando Vásquez González	2026011588
	Anderson Steven Flores Pérez	2026011705
	Luis Gerardo Ramos Martínez	2026010771
	Roberto De Jesus Erazo Ramirez	2026010073

Índice

Introducción 4

Descripción del problema 4

Objetivos 4

- Objetivo general 4
- Objetivos específicos 5

Distribución del equipo 5

- Planificación y cronograma 6

Requerimientos 6

- Roles del sistema 6
- Requerimientos funcionales 6
- Requerimientos no funcionales 7
- Entradas y salidas principales 7

Integración y evolución de los avances anteriores 7

Arquitectura 8

- Modelo de datos 10

Diagrama UML actualizado 10

POO aplicada 12

- Encapsulamiento 12
- Herencia 12
- Polimorfismo 12
- Abstracción 13
- Interfaces 13

Colecciones y genéricos 13

Programación concurrente 14

- Qué proceso se ejecuta concurrentemente 14
- Por qué se utiliza concurrencia 14
- Mecanismo utilizado 14
- Cómo interactúa con la base de datos 15
- Control de problemas de concurrencia 15
- Evidencia de ejecución 16

JDBC 17

- Conexión 17
- DataSource y pool 17

DAO y operaciones CRUD 17
PreparedStatement, ResultSet y cierre de recursos 17
Transacciones 18
Configuración del DataSource 19
Cohesión y acoplamiento 20
Una clase con buena cohesión: NotificacionCorreo 20
Una dependencia que se redujo: services contra interfaces DAO 20
Una decisión de diseño que disminuye el acoplamiento 20
Legibilidad y reutilización 20
Ejemplo antes y después 21
Manejo de excepciones 21
Excepciones propias 21
try, catch, finally, throw y throws 22
Traducción a HTTP 22
Pruebas unitarias 22
Evidencia de ejecución 24
Git y GitHub 24
Conclusiones 26
Bibliografía 27

Introducción

Este documento corresponde a la entrega final del proyecto de Programación II del Grupo 10: un sistema para administrar la Asociación de Desarrollo Comunal (ADESCO) del Residencial Los Maquilishuats, en San Salvador. Recoge la evolución de los dos avances anteriores y describe lo que se incorporó en esta etapa.

En el avance 1 se definió el problema, los roles y un primer modelo de clases centrado en el padrón de socios. En el avance 2 se construyó el sistema funcional: un backend en Spring Boot con arquitectura por capas, un frontend en Next.js con paneles para vecinos y directiva, persistencia en archivos `.dat` y el envío de correos en segundo plano con un pool de hilos.

En la entrega final la persistencia pasa a una base de datos relacional PostgreSQL mediante JDBC, con un `DataSource` configurado como pool de conexiones Tomcat JDBC. La concurrencia ahora trabaja directamente con la base: la generación de las cuotas del mes reparte una tarea por vecino entre varios hilos, y cada tarea toma su propia conexión del pool. Además, se incorporaron excepciones propias con un manejador global, 57 pruebas automatizadas con JUnit 5 y Mockito, y mensajes de error visibles en el frontend.

El orden de las secciones sigue los 18 puntos que pide la guía de la entrega final. Las evidencias de ejecución (pruebas, generación concurrente de cuotas e historial de GitHub) se incluyen en la sección correspondiente.

Descripción del problema

La directiva de una ADESCO administra información que hoy vive dispersa en cuadernos, hojas de cálculo y grupos de chat: el padrón de vecinos, las cuotas mensuales, los gastos, las actas de las asambleas, las solicitudes de los vecinos y los avisos a la comunidad. Esto genera problemas concretos:

- No hay un registro confiable de quién pagó cada mes. El tesorero reconstruye el estado de cuenta a mano.
- Las decisiones sobre proyectos (alumbrado, bacheo, techado de la cancha) se toman sin un registro formal de votos.
- Las solicitudes de los vecinos se pierden en el chat y no queda constancia de la respuesta.
- Los comunicados y actas no tienen un lugar oficial de consulta.
- Cada mes el tesorero debe crear la cuota de cada vecino, una por una. Con 18 vecinos en la semilla del sistema la tarea es repetitiva y fácil de duplicar.

El sistema centraliza esa información con acceso por rol: el vecino consulta y participa; el presidente, el secretario y el tesorero administran solo los módulos que les corresponden.

Objetivos

Objetivo general

Desarrollar un sistema informático para la administración de una asociación comunal que permita a la directiva y a los vecinos gestionar usuarios, proyectos, votaciones, secretaría y tesorería sobre una base de datos relacional, notificando por correo los hechos que competen a cada uno, con el fin de centralizar la organización interna y la transparencia en el manejo de la información de la comunidad.

Objetivos específicos

- **Avance 1.** Diseñar el módulo de registro y gestión de los miembros de la asociación, hoy representados como vecinos y miembros de la directiva.
- **Avance 2.** Implementar el backend en Spring Boot con POO y arquitectura por capas, el frontend en Next.js con paneles por rol y el envío de correos en segundo plano mediante hilos.
- **Entrega final.** Migrar la persistencia a PostgreSQL mediante JDBC y un pool de conexiones configurado, integrar la concurrencia con la base de datos en la generación mensual de cuotas y asegurar la calidad del código con excepciones propias, pruebas automatizadas y control de versiones.

El tercer objetivo específico quedó pendiente en el avance 2 porque la cátedra aún no lo había definido. Se concreta con la guía de la entrega final.

Distribución del equipo

La entrega final la realizan los cuatro integrantes que participaron en el avance 2. Los demás integrantes listados en los avances anteriores no forman parte de esta entrega y se retiran del documento. Se mantienen los roles Scrum y técnicos asignados en el avance 1; por lo reducido del equipo, el trabajo se repartió por módulos.

Tabla 1. Integrantes y roles

Integrante	CIF	Rol Scrum	Rol técnico
Marcos Armando Vásquez González	2026011588	Product Owner (Business Owner)	Analista
Roberto De Jesus Erazo Ramirez	2026010073	Business Analyst	Analista de negocio
Luis Gerardo Ramos Martínez	2026010771	Architect	Arquitecto
Anderson Steven Flores Pérez	2026011705	Developer	Interfaz (frontend)

Reparto real de la entrega final

Tabla 2. Reparto de la entrega final

Integrante	Qué hizo en la entrega final
Luis Gerardo Ramos Martínez	Migración a PostgreSQL: `docker-compose.yml`, `schema.sql`, `DataSourceConfig` con Tomcat JDBC Pool y contraseñas con BCrypt. DAO JDBC de usuarios, votos, solicitudes, actas, comunicados, cuotas, movimientos y bitácora de correos. Excepciones propias y `ApiExceptionHandler`. Generación concurrente de cuotas. Pruebas de servicios, plantillas y controller. Mensajes de error y confirmaciones en el frontend.
Marcos Armando Vásquez González	Revisión y actualización del documento final y del README, diagramas UML, pruebas finales del sistema (pruebas manuales de los flujos de vecino y directiva) y cierre de la entrega.
Anderson Steven Flores Pérez	Migración de proyectos y comentarios a JDBC (`ProyectoDAOJdbcImpl`, `ComentarioDAOJdbcImpl`), sus pruebas de integración y el `Dockerfile` del backend.
Roberto De Jesus Erazo Ramirez	Regla de negocio de presidente único al crear o editar usuarios en `UsuarioServiceImpl`, con seis pruebas unitarias en `UsuarioServiceImplTest`.

Planificación y cronograma

Tabla 3. Actividades de la entrega final

Actividad	Responsable	Fecha
Base de datos en Docker, esquema, pool de conexiones y usuarios con JDBC	Luis Gerardo Ramos Martínez	21 de octubre de 2026
Pruebas de `Roles` y `UsuarioServiceImpl`	Luis Gerardo Ramos Martínez	21 de octubre de 2026
Proyectos y comentarios con JDBC	Anderson Steven Flores Pérez	24 de octubre de 2026
Regla de presidente único	Roberto De Jesus Erazo Ramirez	24 de octubre de 2026
Votos, solicitudes, actas, comunicados, cuotas, movimientos y bitácora con JDBC	Luis Gerardo Ramos Martínez	25 de octubre de 2026
Excepciones propias, cuotas concurrentes, pruebas y mensajes en el frontend	Luis Gerardo Ramos Martínez	25 de octubre de 2026
Documento final, README, diagramas, pruebas finales y entrega	Marcos Armando Vásquez González, con revisión de los cuatro integrantes	Desde el 25 de octubre de 2026

Tabla 4. Cronograma general del proyecto

Entrega	Julio	Agosto	Septiembre	Octubre	Noviembre
Avance 1: 30 jul al 6 ago	■	■			
Avance 2: 11 al 27 sep			■		
Entrega final: desde el 21 oct				■	Defensa

Requerimientos

Roles del sistema

Tabla 5. Roles y funciones

Rol	Funciones en el sistema
Vecino	Iniciar sesión, ver proyectos, votar una vez por proyecto, comentar, enviar solicitudes, leer comunicados y actas publicadas y consultar sus cuotas.
Presidente	Gestionar proyectos y usuarios, escribir en secretaría (actas, solicitudes, comunicados) y en tesorería (cuotas, pagos, gastos y generación de cuotas del mes).
Secretario	Publicar actas, responder solicitudes y publicar comunicados. No escribe en tesorería.
Tesorero	Crear cuotas, generar las cuotas del mes, registrar pagos y gastos y consultar el balance. No publica actas ni comunicados.

Requerimientos funcionales

Tabla 6. Historias de usuario de la entrega final

ID	Historia de usuario	Criterios de aceptación
HU-001	Como directiva, quiero registrar y consultar usuarios para controlar quién accede.	Correo único (también `UNIQUE` en la base). Solo puede existir un presidente. La contraseña se guarda con BCrypt.
HU-002	Como directiva, quiero crear, editar y eliminar proyectos.	El proyecto se guarda con nombre, descripción, estado y fecha, y aparece en el listado.
HU-003	Como vecino, quiero votar una sola vez por proyecto.	Un segundo voto del mismo vecino actualiza el anterior. `UNIQUE (proyecto_id, correo_usuario)` lo garantiza en la base.
HU-004	Como vecino, quiero enviar solicitudes y ver la respuesta.	Presidente o secretario responden; el vecino ve estado y respuesta; se notifica por correo.

ID	Historia de usuario	Criterios de aceptación
HU-005	Como presidente o secretario, quiero publicar actas y comunicados.	El acta y sus acuerdos se guardan en una sola transacción. El vecino solo ve lo publicado.
HU-006	Como presidente o tesorero, quiero registrar cuotas, pagos y gastos.	No se permite una cuota duplicada por vecino y periodo. Marcar una cuota como pagada crea el ingreso en el balance.
HU-007	Como vecino, quiero comentar un proyecto.	No se guardan comentarios vacíos. La directiva los ve en su panel.
HU-008	Como usuario, quiero recibir avisos por correo sin esperar en la pantalla.	El correo se encola tras guardar el dato y se envía en un hilo aparte; cada intento queda en `registro_email`.
HU-009	Como tesorero, quiero generar las cuotas del mes para todos los vecinos con un solo clic.	Se crea una cuota por vecino en paralelo. El resultado informa creadas, ya existentes y fallidas. Repetir el mes no duplica cuotas.
HU-010	Como directiva, quiero ver por qué falló una acción.	El formulario muestra el mensaje del backend (por ejemplo, cuota duplicada) o un aviso si el backend no responde.

Requerimientos no funcionales

- Persistencia en PostgreSQL 16 con JDBC directo (sin JPA ni `JdbcTemplate`), para que el acceso a datos sea explícito.
- Pool de conexiones Tomcat JDBC con parámetros justificados para el tamaño de una ADESCO.
- Errores con un formato JSON uniforme `{ "mensaje": ... }` y el código HTTP correcto (400, 403, 404 o 500).
- El envío de correos nunca bloquea ni revierte la operación de negocio.
- Sesión con JWT en una cookie HttpOnly; contraseñas con BCrypt.
- Entorno reproducible: la base se levanta con `docker compose up -d` y el esquema se aplica solo.
- Pruebas automatizadas ejecutables con `gradlew test`.

Entradas y salidas principales

Tabla 7. Entradas y salidas

Proceso	Entradas	Salidas
Generar cuotas del mes	Periodo (YYYY-MM) y monto	Resumen: creadas, ya existentes y fallidas; un correo de cuota pendiente por cuota creada
Marcar cuota pagada	Identificador de la cuota	Cuota en estado PAGADA, ingreso en movimientos y correo de pago registrado
Votar	Proyecto y sentido del voto	Voto registrado o actualizado; totales para la directiva
Responder solicitud	Estado y respuesta	Solicitud actualizada y correo al vecino (salvo EN_REVISION)
Crear acta	Título, contenido y acuerdos	Acta en borrador con sus acuerdos en `acuerdo_acta`

Integración y evolución de los avances anteriores

La entrega final conserva el diseño de los avances 1 y 2 y lo extiende. La siguiente tabla resume la evolución por aspecto.

Tabla 8. Evolución del proyecto

Aspecto	Avance 1	Avance 2	Entrega final
Modelo	Socio, Cuota y Usuario con rol en texto	`Usuario` abstracta con `Vecino` y `MiembroDirectiva`	Mismo modelo, persistido en la tabla `usuario` con columna `tipo`
Persistencia	Diseño de DAO	Archivos `.dat` con serialización	PostgreSQL con JDBC y pool Tomcat; los `.dat` quedan como respaldo

Aspecto	Avance 1	Avance 2	Entrega final
Concurrencia	No aplica	Pool de hilos para correos	Generación concurrente de cuotas con una conexión por hilo; bitácora de correos en la base
Errores	No aplica	`try/catch` repetido en cada controller	Excepciones propias y `@RestControllerAdvice`
Pruebas	Plan de pruebas inicial	Solo `contextLoads()`	57 pruebas con JUnit 5 y Mockito
Seguridad	Roles definidos	JWT en cookie HttpOnly	Además, contraseñas con BCrypt
Ejecución	Consola	Manual	Base con Docker Compose y README con los pasos

Observaciones de los avances anteriores

- **Sin base de datos (limitación del avance 2).** Resuelta: todas las entidades se guardan en PostgreSQL.
- **Objetivo específico 3 pendiente.** Definido en la sección de objetivos con base en la guía final.
- **Contraseñas en texto plano.** Ahora se guardan con `BCryptPasswordEncoder`.
- **Duplicados validados solo en Java.** La base los impide con restricciones `UNIQUE` en correo, voto y cuota por periodo.
- **`try/catch` repetidos en los controllers.** Reemplazados por el manejador global (ver legibilidad).
- **Bitácora de correos en un `ConcurrentHashMap` sobre `emails.dat`.** Ahora es la tabla `registro_email`; la base resuelve las escrituras concurrentes de los hilos.

Arquitectura

El proyecto es un monorepo con dos aplicaciones. El frontend Next.js (puerto 3000) consume la API REST del backend Spring Boot (puerto 8081). El backend separa responsabilidades en capas y es el único que accede a la base.

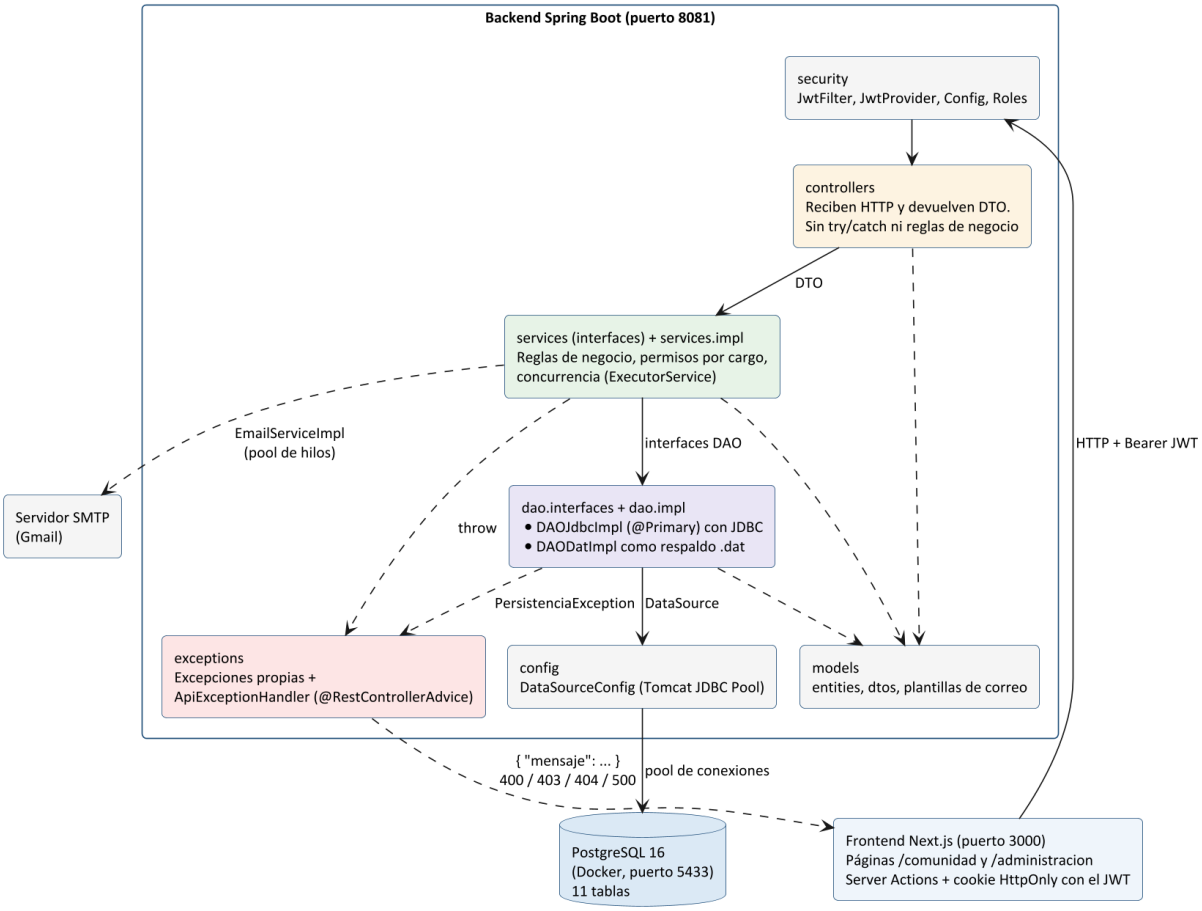


Figura 1. Arquitectura por capas del sistema

Tabla 9. Paquetes del backend

Capa o paquete	Responsabilidad
`controllers`	Reciben la petición HTTP, obtienen el usuario autenticado y delegan en el service. No tienen reglas de negocio ni `try/catch`.
`services` y `services.impl`	Reglas de negocio y permisos por cargo. Dependen de interfaces DAO, nunca de una implementación concreta.
`dao.interfaces` y `dao.impl`	Acceso a datos. Cada interfaz tiene una implementación JDBC marcada con `@Primary` y otra `.dat` de respaldo.
`config`	`DataSourceConfig`: crea el pool de conexiones Tomcat JDBC.
`exceptions`	Excepciones propias y `ApiExceptionHandler`, que las traduce a respuestas HTTP.
`models`	Entidades (lo que se guarda), DTO (lo que entra y sale por la API) y plantillas de correo.
`security`	JWT, filtro de autenticación, configuración de Spring Security y roles.
`utils`	`ArchivoDatUtil`, utilitario genérico de los DAO `.dat`.

Estructura del repositorio

```
AsociacionComunal-Grupo10/
├── docker-compose.yml              PostgreSQL 16 y pgAdmin
├── backend-asociacion-comunal/
│   └── src/main/java/.../backendasociacioncomunal/
│       ├── config/                 DataSourceConfig
│       ├── controllers/            API REST
│       ├── services/  services/impl/
│       ├── dao/interfaces/  dao/impl/  SemillaComunidad
│       ├── exceptions/             Excepciones propias y advice
│       ├── models/entities/  models/dtos/  models/plantillas/
```

```
│       ├── security/  utils/
│   └── src/main/resources/schema.sql
├── web-asociacion-comunal/        Next.js 16, React 19, shadcn/ui
└── avances/                       Documentos de cada entrega
```

Modelo de datos

El esquema está en `schema.sql` y Docker lo ejecuta al crear el volumen de la base. Tiene 11 tablas. La herencia de `Usuario` se resuelve con una sola tabla y una columna `tipo` (VECINO o DIRECTIVA); una restricción `CHECK` exige `numero_casa` para el vecino y `cargo` para la directiva. Los acuerdos de un acta pasan a su propia tabla con `ON DELETE CASCADE`.

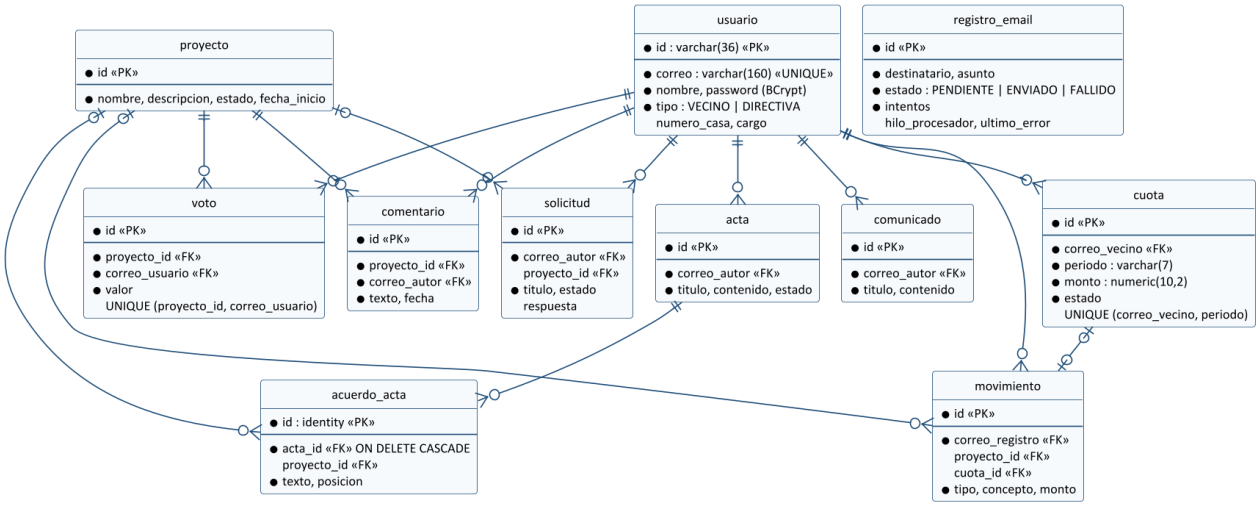


Figura 2. Modelo entidad relación de PostgreSQL

Tres restricciones `UNIQUE` reemplazan validaciones que antes solo existían en Java: `usuario(correo)`, `voto(proyecto_id, correo_usuario)` y `cuota(correo_vecino, periodo)`. Esta última es la que protege la generación concurrente de cuotas.

Diagrama UML actualizado

El diagrama de entidades muestra la jerarquía de `Usuario` y las relaciones entre proyectos, participación, secretaría y tesorería. El de plantillas muestra la jerarquía de correos. El de capas toma el módulo de tesorería como ejemplo de cómo se conectan controller, service, DAO, pool y excepciones.

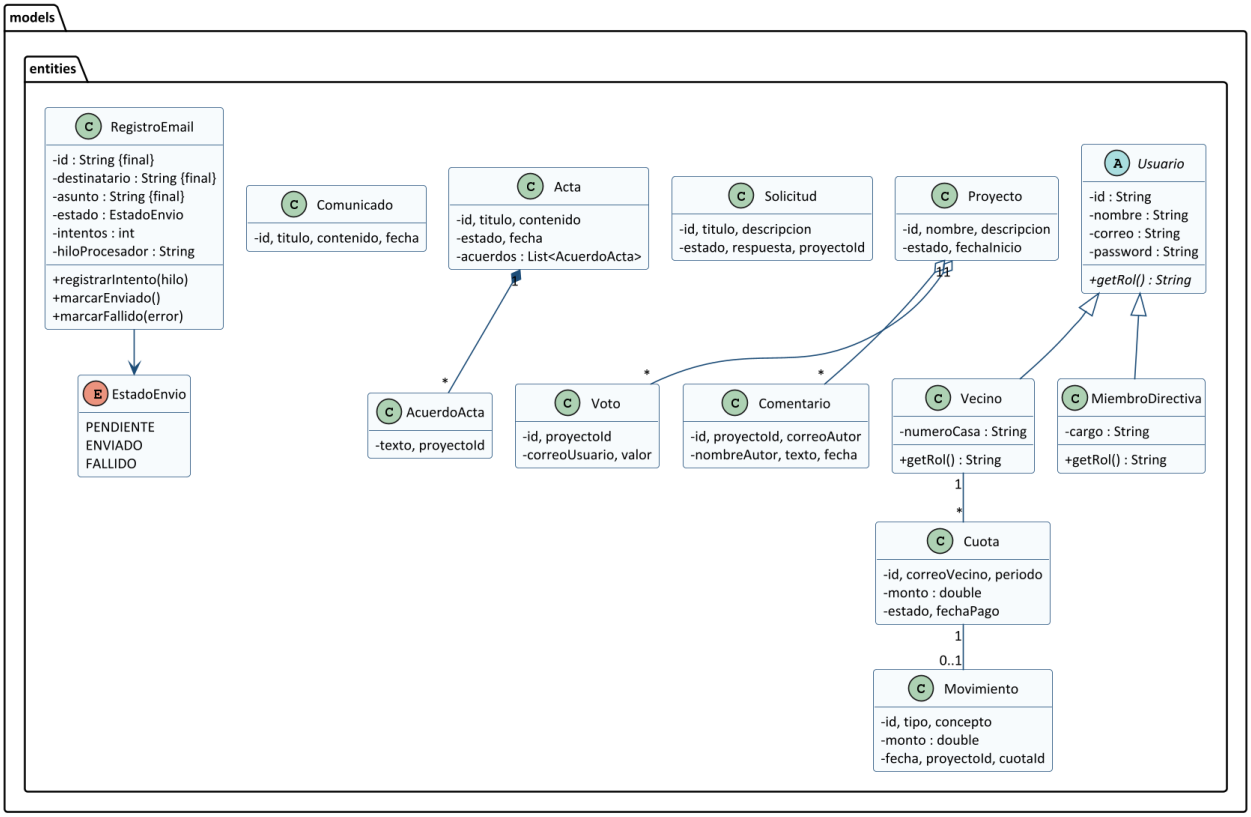


Figura 3. Diagrama de clases de las entidades

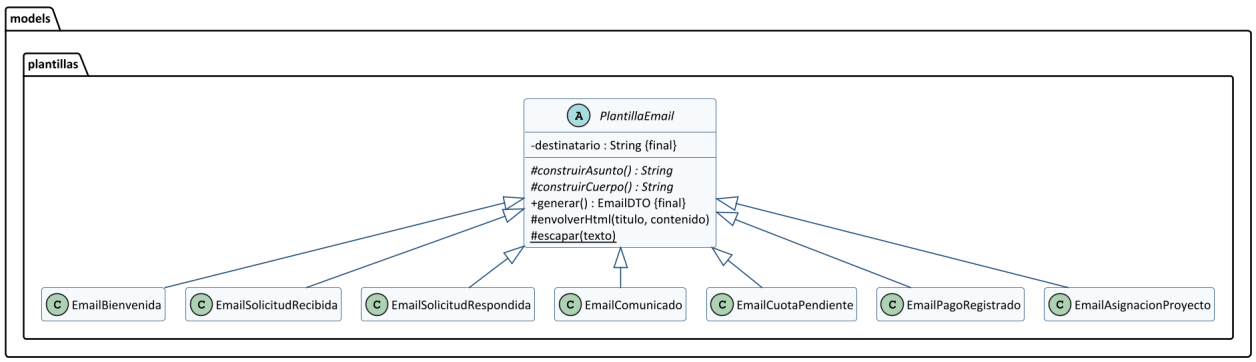


Figura 4. Jerarquía de plantillas de correo

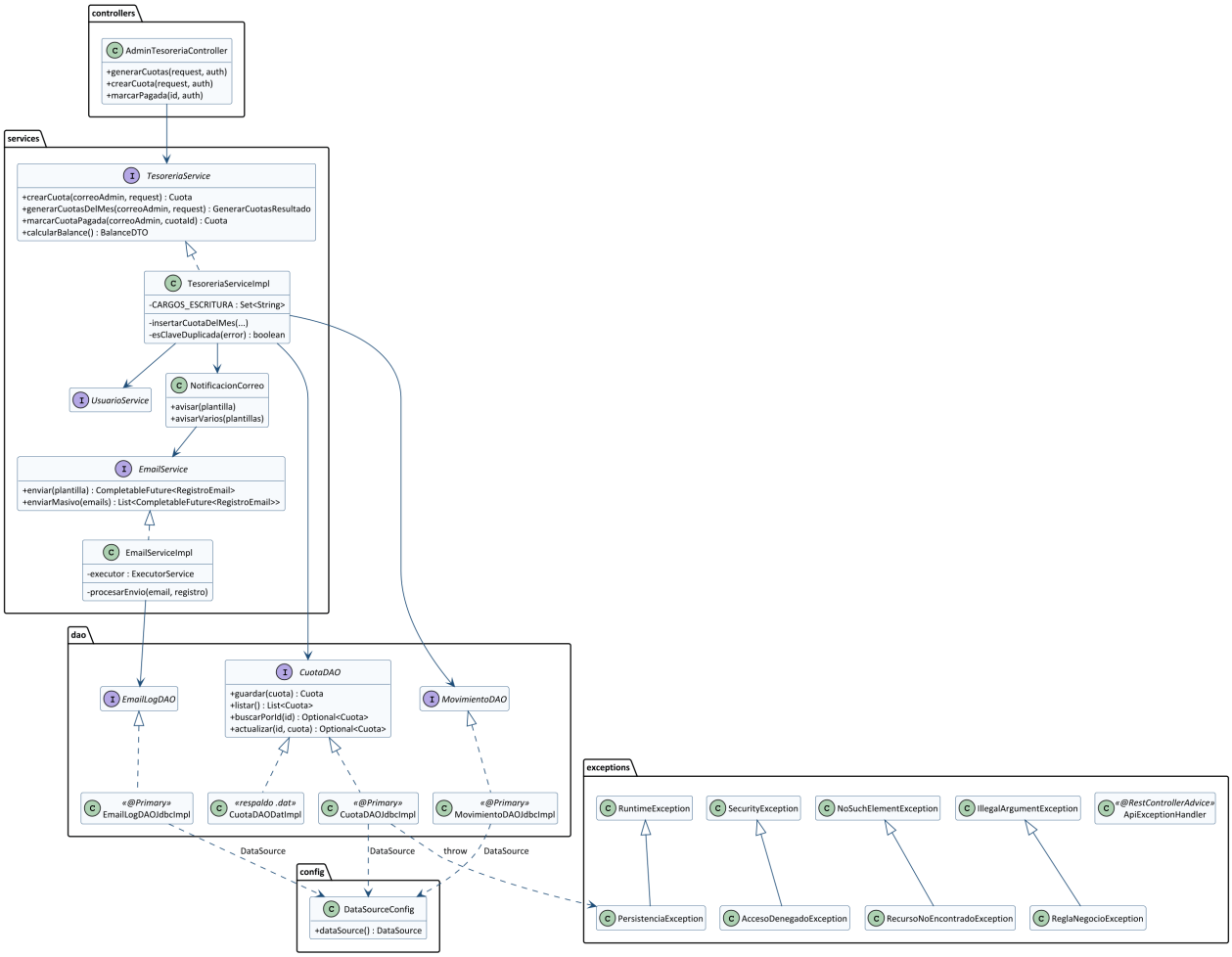


Figura 5. Diagrama de clases por capas (módulo de tesorería)

POO aplicada

Encapsulamiento

Los atributos de las entidades son privados y se acceden por métodos. En `RegistroEmail` el id, el destinatario y el asunto son `final`: se fijan al encolar el correo y no cambian. El estado solo avanza con métodos de negocio (`registrarIntento`, `marcarEnviado`, `marcarFallido`), no con setters libres. En `PlantillaEmail` el destinatario también es `final` y el constructor es `protected`, para que solo las subclases lo usen.

Herencia

`Vecino` y `MiembroDirectiva` extienden `Usuario` y agregan su atributo propio (número de casa o cargo). Las siete plantillas de correo extienden `PlantillaEmail` y reutilizan la estructura HTML común (`envolverHtml`) y el escape de datos (`escapar`). Las excepciones propias también heredan de excepciones del JDK, como se detalla en la sección de excepciones.

Polimorfismo

`Usuario` declara `getRol()` y cada subclase lo resuelve. El filtro JWT, los services y el frontend consultan el rol sin conocer la subclase. En el DAO JDBC el polimorfismo funciona en ambos sentidos: al guardar, `instanceof` decide qué columnas llenar; al leer, la columna `tipo` decide qué subclase construir.

MiembroDirectiva.java

```
public class MiembroDirectiva extends Usuario {
    private String cargo;

    @Override
    public String getRol() {
        return Roles.rolDesdeCargo(cargo);   // PRESIDENTE, SECRETARIO o TESORERO
    }
}
```

Abstracción

`Usuario` es abstracta porque no existe un usuario sin función. `PlantillaEmail` es abstracta porque todo correo nace de un hecho concreto; aplica el patrón método plantilla: `generar()` es `final` y arma el mensaje siempre igual, mientras que cada subclase implementa `construirAsunto()` y `construirCuerpo()`.

PlantillaEmail.java

```
public abstract class PlantillaEmail {
    private final String destinatario;

    protected abstract String construirAsunto();
    protected abstract String construirCuerpo();

    public final EmailDTO generar() {
        return new EmailDTO(destinatario, construirAsunto(), construirCuerpo(), esHtml());
    }
}
```

Interfaces

Cada service y cada DAO se define como interfaz (`TesoreriaService`, `CuotaDAO`, `EmailService`, etc.). Los services reciben interfaces por constructor, y Spring inyecta la implementación JDBC porque está marcada con `@Primary`. Así el mismo `TesoreriaServiceImpl` funcionó con archivos `.dat` en el avance 2 y funciona con PostgreSQL ahora, sin cambiar una línea de su lógica de persistencia.

Una interfaz, dos implementaciones

```
public interface CuotaDAO {
    Cuota guardar(Cuota cuota);
    List<Cuota> listar();
    Optional<Cuota> buscarPorId(String id);
    Optional<Cuota> actualizar(String id, Cuota cuota);
}

@Repository @Primary
public class CuotaDAOJdbcImpl implements CuotaDAO { ... }   // PostgreSQL
public class CuotaDAODatImpl  implements CuotaDAO { ... }   // respaldo .dat
```

Colecciones y genéricos

Tabla 10. Colecciones y genéricos utilizados

Colección o tipo genérico	Dónde	Por qué
`List<T>` / `ArrayList`	Resultados de todos los DAO	Conserva el orden del `ORDER BY` de la consulta.
`Set<String>` con `Set.of`	`CARGOS_ESCRITURA`, `TIPOS_MOVIMIENTO`	Verificación de pertenencia en una sola operación e inmutable.

Colección o tipo genérico	Dónde	Por qué
`Optional<T>`	`buscarPorId`, `buscarPorCorreo`	Obliga al service a decidir qué hacer si no existe, en lugar de devolver `null`.
`Map<String, String>`	Cuerpo de error en `ApiExceptionHandler`	Respuesta `{ "mensaje": ... }` uniforme.
`LinkedHashMap<String, List<AcuerdoActa>>`	`ActaDAOJdbcImpl`	Agrupa los acuerdos por acta en una sola consulta y respeta el orden.
`List<Future<?>>`	Generación de cuotas	Espera el fin de cada tarea concurrente y propaga sus errores.
`AtomicInteger`	Contadores de creadas, existentes y fallidas	Incrementos seguros desde varios hilos sin `synchronized`.
`CompletableFuture<RegistroEmail>`	`EmailService`	El envío continúa en otro hilo y el llamador no espera.
`ConcurrentHashMap`	`EmailLogDAODatImpl` (respaldo)	Mapa seguro para varios hilos cuando la bitácora vivía en archivo.
Métodos genéricos `<T>`	`ArchivoDatUtil.guardarDatos` y `leerDatos`	Un solo utilitario sirve para cualquier entidad serializable.

Filtrar vecinos con streams (TesoreriaServiceImpl)

```
List<Vecino> vecinos = usuarioService.listarUsuarios().stream()
        .filter(Vecino.class::isInstance)
        .map(Vecino.class::cast)
        .toList();
```

Programación concurrente

Qué proceso se ejecuta concurrentemente

La **generación de las cuotas del mes**. El tesorero (o el presidente) indica un periodo y un monto, y el sistema crea una cuota pendiente para cada vecino del residencial, avisándole por correo. Hay un segundo proceso concurrente que ya existía y ahora también escribe en la base: el **envío de correos** del `EmailServiceImpl`.

Por qué se utiliza concurrencia

Cada cuota es independiente de las demás: no hay orden entre vecinos ni datos compartidos entre inserciones. Insertarlas en paralelo reduce el tiempo total cuando crece el número de vecinos y aprovecha que el pool ya tiene varias conexiones abiertas. En el caso del correo, una conexión SMTP tarda segundos; si se enviara en el hilo de la petición, el usuario esperaría en pantalla.

Mecanismo utilizado

- `ExecutorService` con `Executors.newFixedThreadPool(min(4, vecinos))`: como máximo cuatro hilos, por debajo de `maxActive = 10`, para dejar conexiones libres a las peticiones web y a los hilos de correo.
- Una tarea (`Runnable` enviado con `submit`) por vecino; el service guarda los `Future` y llama a `get()` en cada uno.
- `AtomicInteger` para los contadores compartidos.
- `shutdown()` y `awaitTermination(30 s)` en un bloque `finally`, para no dejar hilos vivos aunque falle una tarea.

TesoreriaServiceImpl.generarCuotasDelMes (resumido)

```
int hilos = Math.max(1, Math.min(4, vecinos.size()));
ExecutorService executor = Executors.newFixedThreadPool(hilos);
try {
    List<Future<?>> tareas = new ArrayList<>();
    for (Vecino vecino : vecinos) {
        tareas.add(executor.submit(() ->
```

```
                insertarCuotaDelMes(vecino, periodo, request.monto(), creadas, existentes, fallidas)));
    }
    for (Future<?> tarea : tareas) {
        tarea.get();
    }
} catch (InterruptedException e) {
    Thread.currentThread().interrupt();
    throw new PersistenciaException("Se interrumpio la generacion de cuotas", e);
} catch (ExecutionException e) {
    throw new PersistenciaException("Fallo una tarea de generacion de cuotas", e.getCause());
} finally {
    executor.shutdown();
    if (!executor.awaitTermination(30, TimeUnit.SECONDS)) executor.shutdownNow();
}
```

Cómo interactúa con la base de datos

Cada tarea llama a `cuotaDAO.guardar(cuota)`. Dentro del DAO, `dataSource.getConnection()` toma una conexión **propia** del pool; ninguna `Connection` se comparte entre hilos. El `INSERT` se ejecuta con `PreparedStatement.executeUpdate()` y el try-with-resources devuelve la conexión al pool al terminar, haya error o no.

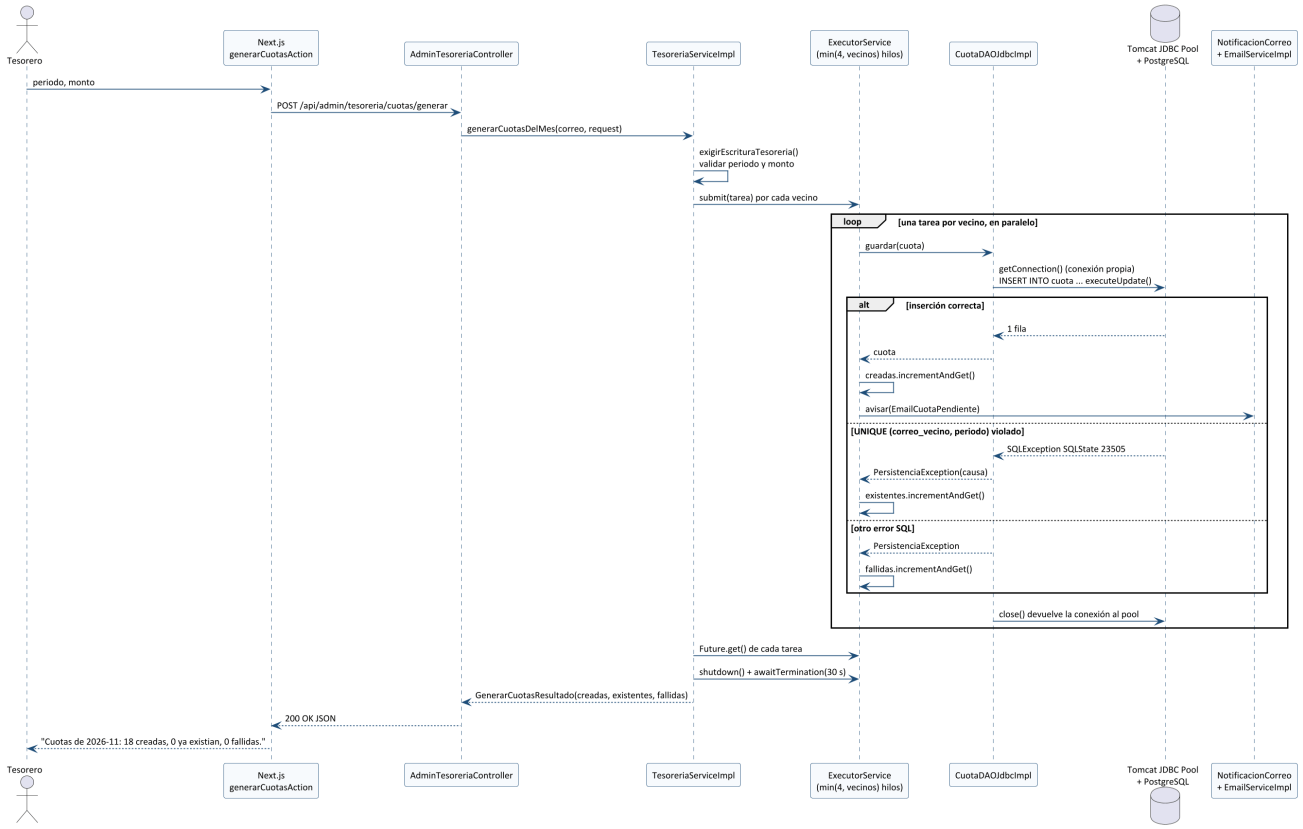


Figura 6. Secuencia de la generación concurrente de cuotas

Control de problemas de concurrencia

- **Duplicados.** Si dos tesoreros generan el mismo mes al mismo tiempo, o se pulsa dos veces el botón, la restricción `UNIQUE (correo_vecino, periodo)` rechaza la segunda inserción. La tarea reconoce el `SQLState 23505` en la causa de la `PersistenciaException` y lo cuenta como "ya existía" en lugar de error.
- **Conexiones.** Una conexión por tarea y siempre cerrada con try-with-resources. `removeAbandoned` recupera cualquier conexión que quedara retenida más de 60 s.

- **Contadores.** `AtomicInteger` evita condiciones de carrera al sumar resultados.
- **Errores aislados.** Un fallo en un vecino no detiene a los demás; se cuenta como fallida. Un fallo del correo nunca revierte la cuota guardada (`NotificacionCorreo` lo registra y sigue).
- **Interrupción.** Se restaura la bandera de interrupción del hilo y se apaga el pool.

Detección de la violación de UNIQUE

```
private boolean esClaveDuplicada(Throwable error) {
    Throwable actual = error;
    while (actual != null) {
        if (actual instanceof SQLException sql && "23505".equals(sql.getSQLState())) {
            return true;
        }
        actual = actual.getCause();
    }
    return false;
}
```

Evidencia de ejecución

Primera ejecución para el periodo 2026-10 con los 18 vecinos de la semilla: se crearon las 18 cuotas. Segunda ejecución del mismo periodo: la base rechazó las 18 inserciones por la restricción `UNIQUE` y el sistema las reportó como ya existentes, sin duplicar ninguna.

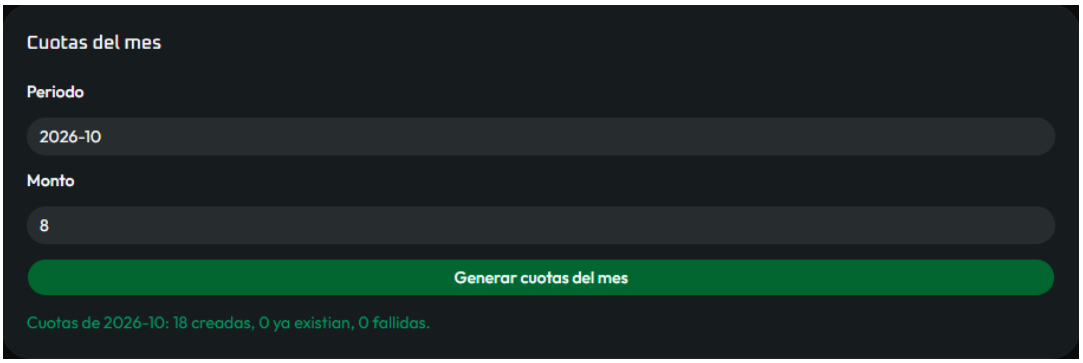


Figura 7. Primera generación de cuotas del mes

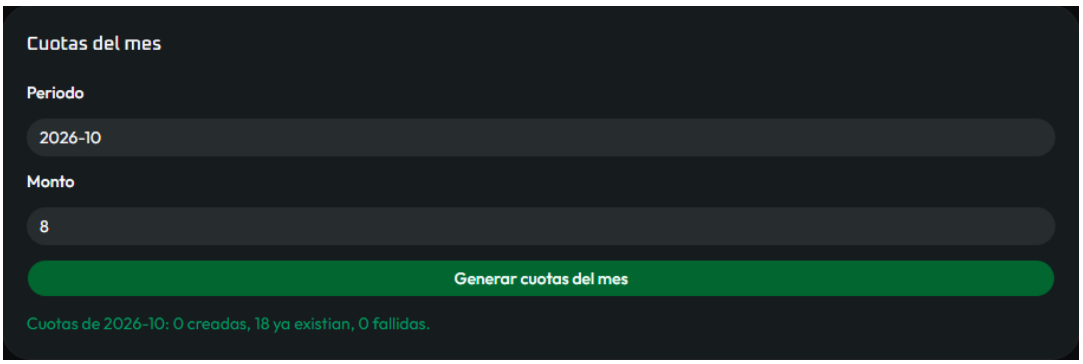


Figura 8. Segunda generación del mismo periodo: control de duplicados

Los correos de esas 18 cuotas se procesaron en el pool de tres hilos del `EmailServiceImpl`, y cada hilo dejó su rastro en la tabla `registro_email`. En la prueba el servidor SMTP estaba desactivado a propósito, por eso los envíos terminan en FALLIDO después de tres intentos, sin afectar a las cuotas ya guardadas:

Bitácora de correos escrita por los hilos

```
SELECT hilo_procesador, estado, count(*) AS correos, max(intentos) AS intentos
FROM registro_email WHERE asunto ILIKE '%2026-10%' GROUP BY 1, 2;

 hilo_procesador | estado  | correos | intentos
-----------------+---------+---------+----------
 email-sender-1  | FALLIDO |       6 |        3
 email-sender-2  | FALLIDO |       6 |        3
 email-sender-3  | FALLIDO |       6 |        3
```

JDBC

Conexión

La URL, el usuario y la contraseña se leen de `application.properties` (`app.db.url`, `app.db.username`, `app.db.password`). La base corre en Docker y se publica en el puerto 5433 para no chocar con un PostgreSQL instalado en la máquina: `jdbc:postgresql://localhost:5433/asociacion_comunal`.

DataSource y pool

`DataSourceConfig` expone un `@Bean` de tipo `org.apache.tomcat.jdbc.pool.DataSource`. Los DAO lo reciben por constructor como `javax.sql.DataSource`, es decir, dependen de la interfaz estándar de JDBC y no del pool concreto. Pedir una conexión al pool no abre un socket nuevo: entrega una conexión ya abierta y validada, y `close()` la devuelve en lugar de cerrarla.

DAO y operaciones CRUD

Tabla 11. Implementaciones JDBC

DAO JDBC	Tabla	Operaciones
`UsuarioDAOJdbcImpl`	usuario	Insertar, consultar (por id, correo y todos), actualizar y eliminar
`ProyectoDAOJdbcImpl`	proyecto	Insertar, consultar, actualizar y eliminar
`VotoDAOJdbcImpl`	voto	Insertar o actualizar el voto, consultar por proyecto y por vecino
`ComentarioDAOJdbcImpl`	comentario	Insertar y consultar
`SolicitudDAOJdbcImpl`	solicitud	Insertar, consultar y actualizar (respuesta)
`ActaDAOJdbcImpl`	acta, acuerdo_acta	Insertar y actualizar en transacción, consultar con sus acuerdos
`ComunicadoDAOJdbcImpl`	comunicado	Insertar y consultar
`CuotaDAOJdbcImpl`	cuota	Insertar, consultar y actualizar (pago)
`MovimientoDAOJdbcImpl`	movimiento	Insertar y consultar
`EmailLogDAOJdbcImpl`	registro_email	Insertar, consultar y actualizar desde los hilos de envío

Cada DAO JDBC siembra su tabla con `SemillaComunidad` si la encuentra vacía (`@PostConstruct`). `@DependsOn` garantiza el orden de la semilla para respetar las llaves foráneas: primero usuarios y proyectos, después lo que depende de ellos.

PreparedStatement, ResultSet y cierre de recursos

Todas las consultas usan `PreparedStatement` con parámetros `?`, lo que evita inyección SQL. Las lecturas usan `executeQuery()` y recorren el `ResultSet`; las escrituras usan `executeUpdate()` y revisan cuántas filas cambiaron.

`Connection`, `PreparedStatement` y `ResultSet` se declaran en try-with-resources, así se cierran en orden inverso aunque ocurra una excepción.

CuotaDAOJdbcImpl: consulta y actualización

```
@Override
public Optional<Cuota> buscarPorId(String id) {
    try (Connection connection = dataSource.getConnection();
         PreparedStatement statement = connection.prepareStatement(SELECT_BY_ID)) {
        statement.setString(1, id);
        try (ResultSet resultSet = statement.executeQuery()) {
            if (!resultSet.next()) {
                return Optional.empty();
            }
            return Optional.of(mapear(resultSet));
        }
    } catch (SQLException e) {
        throw new PersistenciaException("No se pudo buscar la cuota", e);
    }
}

@Override
public Optional<Cuota> actualizar(String id, Cuota cuota) {
    try (Connection connection = dataSource.getConnection();
         PreparedStatement statement = connection.prepareStatement(UPDATE_SQL)) {
        ... // setString / setBigDecimal de cada columna
        if (statement.executeUpdate() != 1) {
            return Optional.empty();
        }
        return Optional.of(cuota);
    } catch (SQLException e) {
        throw new PersistenciaException("No se pudo actualizar la cuota", e);
    }
}
```

Transacciones

Un acta y sus acuerdos se guardan juntos o no se guardan. `ActaDAOJdbcImpl` desactiva el autocommit, inserta en `acta` y en `acuerdo_acta` con la misma conexión, y hace `commit` o `rollback`. El `finally` restaura el autocommit antes de que la conexión vuelva al pool, para que la próxima petición no herede una transacción abierta.

ActaDAOJdbcImpl.guardar

```
try (Connection connection = dataSource.getConnection()) {
    connection.setAutoCommit(false);
    try {
        insertarActa(connection, acta);
        insertarAcuerdos(connection, acta.getId(), acta.getAcuerdos());
        connection.commit();
    } catch (SQLException e) {
        connection.rollback();
        throw e;
    } finally {
        connection.setAutoCommit(true);
    }
} catch (SQLException e) {
    throw new PersistenciaException("No se pudo guardar el acta", e);
}
```

Configuración del DataSource

El pool se dimensionó para una ADESCO: 21 usuarios en la semilla (3 de la directiva y 18 vecinos), pocas escrituras simultáneas y dos procesos concurrentes que piden conexiones (hasta 4 hilos de cuotas y 3 de correo). Los valores aplicados son los de `DataSourceConfig`:

DataSourceConfig.dataSource()

```
dataSource.setInitialSize(5);
dataSource.setMinIdle(5);
dataSource.setMaxIdle(10);
dataSource.setMaxActive(10);
dataSource.setMaxWait(10000);

dataSource.setValidationQuery("SELECT 1");
dataSource.setTestOnBorrow(true);
dataSource.setTestOnReturn(false);
dataSource.setTestWhileIdle(true);
dataSource.setTestOnConnect(true);
dataSource.setValidationInterval(30000);

dataSource.setTimeBetweenEvictionRunsMillis(30000);
dataSource.setMinEvictableIdleTimeMillis(60000);
dataSource.setRemoveAbandoned(true);
dataSource.setRemoveAbandonedTimeout(60);
```

Tabla 12. Parámetros del pool y su justificación

Parámetro	Valor	Propósito	Justificación para este sistema
initialSize	5	Conexiones creadas al arrancar	Al iniciar, la semilla y el primer login ya encuentran conexiones listas.
minIdle	5	Mínimo de conexiones inactivas	Igual a `initialSize`: el pool no baja de 5 y no hay que abrir conexiones en frío en horas tranquilas.
maxIdle	10	Máximo de conexiones inactivas	Igual a `maxActive`: después de un pico no se cierran conexiones que probablemente se vuelvan a usar.
maxActive	10	Máximo de conexiones en uso	Cubre el peor caso real: 4 hilos de cuotas, 3 de correo y algunas peticiones web. Más conexiones solo consumirían memoria en PostgreSQL.
maxWait	10000 ms	Espera máxima por una conexión	Si el pool se agota, la petición falla a los 10 s con un error claro en lugar de quedar colgada.
validationQuery	SELECT 1	Consulta de validación	La consulta más barata posible para PostgreSQL.
testOnBorrow	true	Validar al pedir la conexión	Nunca se entrega a un service una conexión que el servidor cerró.
testOnReturn	false	Validar al devolverla	Validar al pedirla ya alcanza; validar también al devolver duplica el costo.
testWhileIdle	true	Validar las inactivas	Detecta conexiones cortadas por reinicios del contenedor de Docker.
testOnConnect	true	Validar al crear la conexión	Una URL o credencial mal configurada se detecta al arrancar, no en la primera petición.
validationInterval	30000 ms	Intervalo mínimo entre validaciones de una conexión	Durante la generación de cuotas la misma conexión se pide muchas veces; no se repite `SELECT 1` más de una vez cada 30 s.
timeBetweenEvictionRunsMillis	30000 ms	Frecuencia de revisión del pool	Cada 30 s se revisan inactivas y abandonadas, un ritmo bajo para el tráfico de una ADESCO.
minEvictableIdleTimeMillis	60000 ms	Inactividad antes de expulsar	Una conexión quieta más de un minuto puede cerrarse (respetando `minIdle`).
removeAbandoned	true	Recuperar conexiones abandonadas	Red de seguridad si algún código no devolviera la conexión al pool.
removeAbandonedTimeout	60 s	Tiempo para considerarla abandonada	Ninguna operación del sistema tarda más de unos segundos; 60 s deja margen sin cortar operaciones legítimas.

A diferencia del ejemplo de la guía, `maxActive` es 10 y no 20: con un solo residencial y como máximo siete hilos de trabajo, 10 conexiones cubren el pico y evitan reservar recursos que la base nunca usaría. `maxIdle` igual a `maxActive` evita abrir y cerrar conexiones después de cada generación de cuotas.

Cohesión y acoplamiento

Una clase con buena cohesión: NotificacionCorreo

`NotificacionCorreo` tiene una sola responsabilidad: encolar avisos de negocio sin que un problema de correo afecte a la operación ya guardada. No arma mensajes (eso lo hacen las plantillas), no conoce SMTP (eso es de `EmailServiceImpl`) y no conoce reglas de negocio. Antes, cada service repetía el mismo `try/catch` alrededor de `emailService.enviar`; ahora `TesoreriaServiceImpl`, `SecretariaServiceImpl` y `UsuarioServiceImpl` llaman a `notificacionCorreo.avisar(plantilla)`.

Una dependencia que se redujo: services contra interfaces DAO

Los services dependen de `CuotaDAO`, `MovimientoDAO` o `UsuarioService`, nunca de `CuotaDAOJdbcImpl`. Gracias a eso, la migración de `.dat` a PostgreSQL consistió en escribir una implementación nueva por interfaz y marcarla con `@Primary`: los services y los controllers no cambiaron por la migración. En las pruebas, la misma decisión permite reemplazar el DAO por un mock de Mockito.

Una decisión de diseño que disminuye el acoplamiento

Los DAO reciben `javax.sql.DataSource` por constructor, no la clase del pool de Tomcat ni una URL. Cambiar Tomcat JDBC por HikariCP, o PostgreSQL por otro motor, se resuelve en `DataSourceConfig` sin tocar los DAO. Del mismo modo, las excepciones de JDBC no salen del DAO: se envuelven en `PersistenciaException`, así los services y controllers no dependen de `java.sql`.

Legibilidad y reutilización

Se revisó el código con la lista que propone la guía. Los cambios principales fueron:

Tabla 13. Revisión de legibilidad

Aspecto revisado	Mejora aplicada
Código duplicado	Se eliminaron los `try/catch` repetidos en cinco controllers: 158 líneas eliminadas y 52 agregadas, 106 líneas menos en total.
Responsabilidades	`NotificacionCorreo` concentra el encolado de avisos; `ApiExceptionHandler` concentra la traducción de errores.
Organización de paquetes	Nuevos paquetes `config` y `exceptions`; DAO separados en `interfaces` e `impl`.
Reutilización	`mapear(ResultSet)` y constantes `COLUMNAS`, `INSERT_SQL` y `SELECT_ALL` por DAO; `escapar()` y `envolverHtml()` compartidos por las plantillas.
Nombres	Métodos que dicen la regla que validan: `exigirEscrituraTesoreria`, `esClaveDuplicada`, `insertarCuotaDelMes`, `validarUnicoPresidente`.
Métodos grandes	La generación de cuotas separa la coordinación (`generarCuotasDelMes`) del trabajo de cada hilo (`insertarCuotaDelMes`).
Tipos de retorno	Los controllers pasaron de `ResponseEntity<?>` a tipos concretos como `ResponseEntity<Cuota>`.

Ejemplo antes y después

Antes, cada método del controller atrapaba las mismas excepciones genéricas y armaba la misma respuesta. El patrón se repetía en todos los endpoints de tesorería, secretaría, proyectos y usuarios:

Antes: AdminTesoreriaController

```
@PostMapping("/cuotas")
public ResponseEntity<?> crearCuota(@RequestBody CuotaRequestDTO request, Authentication
authentication) {
    try {
        Cuota creada = tesoreriaService.crearCuota(authentication.getName(), request);
        return ResponseEntity.status(HttpStatus.CREATED).body(creada);
    } catch (IllegalArgumentException e) {
        return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(Map.of("mensaje", e.getMessage()));
    } catch (NoSuchElementException e) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(Map.of("mensaje", e.getMessage()));
    } catch (SecurityException e) {
        return ResponseEntity.status(HttpStatus.FORBIDDEN).body(Map.of("mensaje", e.getMessage()));
    }
}
```

Después, el controller solo describe el caso feliz y el tipo de retorno es explícito. La traducción a HTTP vive en un único lugar:

Después: AdminTesoreriaController

```
@PostMapping("/cuotas")
public ResponseEntity<Cuota> crearCuota(@RequestBody CuotaRequestDTO request, Authentication
authentication) {
    Cuota creada = tesoreriaService.crearCuota(authentication.getName(), request);
    return ResponseEntity.status(HttpStatus.CREATED).body(creada);
}
```

Manejo de excepciones

Excepciones propias

Tabla 14. Jerarquía de excepciones propias

Excepción	Hereda de	Cuándo se lanza	HTTP
`ReglaNegocioException`	`IllegalArgumentException`	Dato inválido o regla violada: cuota duplicada, monto en cero, segundo presidente.	400
`RecursoNoEncontradoException`	`NoSuchElementException`	La cuota, solicitud, usuario o vecino no existe.	404
`AccesoDenegadoException`	`SecurityException`	El cargo no puede escribir en ese módulo (por ejemplo, el secretario en tesorería).	403
`PersistenciaException`	`RuntimeException`	Envuelve una `SQLException` en los DAO, conservando la causa.	500

Heredar de las excepciones estándar mantiene la compatibilidad con el código que ya atrapaba `IllegalArgumentException` o `SecurityException`, y a la vez permite distinguir un error de negocio propio de uno inesperado del JDK.

try, catch, finally, throw y throws

- **try y catch.** En los DAO, cada `catch (SQLException e)` traduce el error de JDBC a `PersistenciaException`. En `NotificacionCorreo`, el `catch (RuntimeException ex)` tiene una razón técnica: un fallo al encolar un correo no debe revertir la cuota o la solicitud que ya se guardó, así que se registra en el log y la operación continúa.
- **finally.** En `ActaDAOJdbcImpl` restaura el autocommit antes de devolver la conexión; en la generación de cuotas y en `EmailServiceImpl` apaga el pool de hilos; en `procesarEnvio` guarda el resultado del envío en `registro_email` pase lo que pase.
- **throw.** Los services lanzan `ReglaNegocioException`, `RecursoNoEncontradoException` o `AccesoDenegadoException` con un mensaje que el frontend muestra tal cual.
- **throws.** Los métodos auxiliares de JDBC declaran `throws SQLException` (`mapear(ResultSet)`, `bind`, `insertarActa`) y dejan que el método público decida cómo traducirla.
- **Excepciones de JDBC.** `SQLException` nunca sale de la capa DAO. Su `SQLState` se conserva dentro de la causa, lo que permite reconocer el `23505` de clave duplicada en la concurrencia.

Traducción a HTTP

ApiExceptionHandler.java (resumido)

```
@RestControllerAdvice
public class ApiExceptionHandler {

    @ExceptionHandler(ReglaNegocioException.class)
    public ResponseEntity<Map<String, String>> regla(ReglaNegocioException ex) {
        return respuesta(HttpStatus.BAD_REQUEST, ex.getMessage());
    }

    @ExceptionHandler(AccesoDenegadoException.class)
    public ResponseEntity<Map<String, String>> acceso(AccesoDenegadoException ex) {
        return respuesta(HttpStatus.FORBIDDEN, ex.getMessage());
    }

    @ExceptionHandler(PersistenciaException.class)
    public ResponseEntity<Map<String, String>> persistencia(PersistenciaException ex) {
        return respuesta(HttpStatus.INTERNAL_SERVER_ERROR, ex.getMessage());
    }
    ...
}
```

En el frontend, las Server Actions leen el campo `mensaje` y el componente `ActionForm` lo muestra debajo del formulario. Si el backend no responde, el usuario ve "No se pudo contactar el backend. Revisa que este corriendo en el puerto 8081." en lugar de un fallo silencioso.

Pruebas unitarias

Las pruebas usan JUnit 5 y Mockito (incluidos en los starters de prueba de Spring Boot) y verifican resultados con `assertEquals`, `assertThrows`, `verify` y `jsonPath`; no se limitan a ejecutar métodos. Se agrupan en tres tipos:

- **Unitarias de service, utilidades y reglas** con Mockito: el DAO y el correo se reemplazan por mocks, así la prueba no necesita base de datos.
- **De controller** con `@WebMvcTest` y MockMvc: levantan solo la capa web con el `ApiExceptionHandler` y verifican códigos y mensajes HTTP.
- **De integración de DAO** con `@SpringBootTest`: ejecutan SQL real contra PostgreSQL, por eso requieren la base de Docker levantada.

Tabla 15. Pruebas automatizadas (57 en total)

Clase de prueba	Tipo	Pruebas	Qué verifica
`UsuarioServiceImplTest`	Service	12	Autenticación con BCrypt, correo duplicado, alta con aviso de bienvenida, presidente único al crear y editar, errores 404.
`TesoreriaServiceImplTest`	Service	6	Crear cuota y avisar, duplicada, cuota a quien no es vecino, pago que crea el ingreso, permiso por cargo y generación concurrente (2 creadas, 1 duplicada por SQLState 23505).
`SecretariaServiceImplTest`	Service	3	Solicitud que avisa a presidente y secretario, EN_REVISION sin correo y solicitud inexistente.
`NotificacionCorreoTest`	Service	2	Correo vacío no se encola; un fallo al encolar no se propaga.
`RolesTest`	Utilidad	4	`rolDesdeCargo` con cargos válidos, vacío y desconocido.
`PlantillaEmailTest`	Validación	1	`escapar()` neutraliza `<script>` en el cuerpo del correo.
`AdminTesoreriaControllerTest`	Controller	3	200 en el caso feliz, 400 con mensaje por regla de negocio y 403 sin cargo.
`*DAOJdbcImplTest` (9 clases)	Integración	25	Inserción, consulta, actualización y semilla de cada DAO contra PostgreSQL.
`BackendAsociacionComunalApplicationTests`	Contexto	1	La aplicación completa arranca con el pool y la base.

Prueba de la generación concurrente con Mockito

```
@Test
void generarCuotas_cuentaCreadasYDuplicadas() {
    ... // tres vecinos; el DAO mockeado rechaza a uno con SQLState 23505
    when(cuotaDAO.guardar(any(Cuota.class))).thenAnswer(invocation -> {
        Cuota cuota = invocation.getArgument(0);
        if ("ana.guzman@adesco.com".equals(cuota.getCorreoVecino())) {
            throw new PersistenciaException("duplicada", new SQLException("duplicate", "23505"));
        }
        return cuota;
    });

    GenerarCuotasResultado resultado = service.generarCuotasDelMes(
            "tesorero@adesco.com", new GenerarCuotasRequestDTO("2026-11", 8.0));

    assertEquals(2, resultado.creadas());
    assertEquals(1, resultado.existentes());
    assertEquals(0, resultado.fallidas());
    verify(cuotaDAO, times(3)).guardar(any());
    verify(notificacionCorreo, times(2)).avisar(any());
}
```

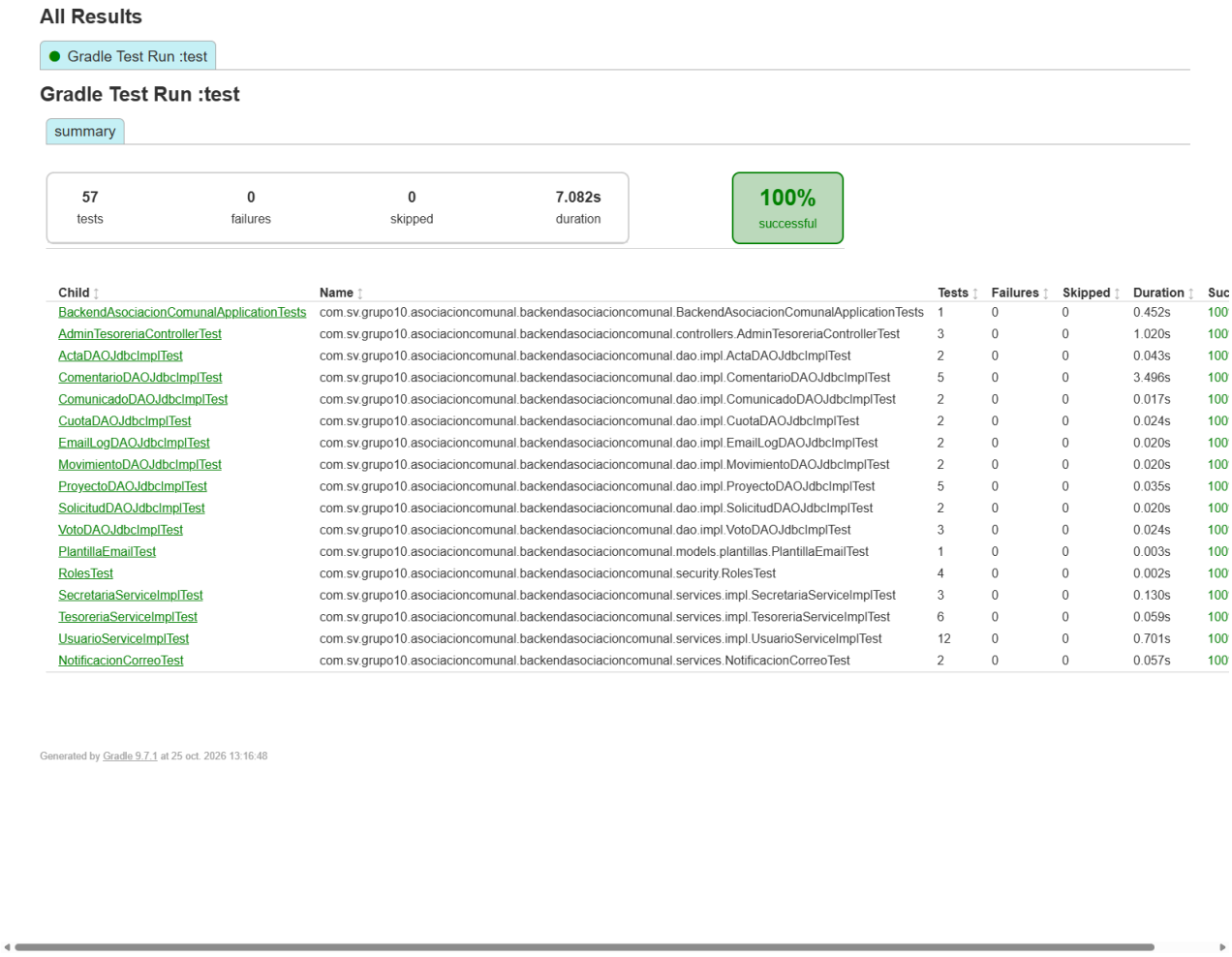
Prueba de controller con MockMvc

```
@Test
void crearCuota_devuelve400CuandoLaReglaFalla() throws Exception {
    when(tesoreriaService.crearCuota(eq("tesorero@adesco.com"), any()))
            .thenThrow(new ReglaNegocioException("Ya existe una cuota para ese vecino en el periodo"));

    mockMvc.perform(post("/api/admin/tesoreria/cuotas")
                    .with(csrf())
                    .contentType(MediaType.APPLICATION_JSON)
                    .content("{\"correoVecino\":\"vecino@adesco.com\",\"periodo\":\"2026-
10\",\"monto\":8}"))
            .andExpect(status().isBadRequest())
            .andExpect(jsonPath("$.mensaje").value("Ya existe una cuota para ese vecino en el
periodo"));
}
```

Evidencia de ejecución

Ejecución de `gradlew test` desde `backend-asociacion-comunal`, con la base de Docker levantada: 57 pruebas, 0 fallos, 100 % exitosas. El reporte queda en `build/reports/tests/test/index.html`.



All Results

Gradle Test Run :test

Gradle Test Run :test

summary

57 tests	0 failures	0 skipped	7.082s duration	100% successful

Child	Name	Tests	Failures	Skipped	Duration	Suc
BackendAsociacionComunalApplicationTests	com.sv.grupo10.asociacioncomunal.backendasociacioncomunal.BackendAsociacionComunalApplicationTests	1	0	0	0.452s	100
AdminTesoreriaControllerTest	com.sv.grupo10.asociacioncomunal.backendasociacioncomunal.controllers.AdminTesoreriaControllerTest	3	0	0	1.020s	100
ActaDAOJdbcImplTest	com.sv.grupo10.asociacioncomunal.backendasociacioncomunal.dao.impl.ActaDAOJdbcImplTest	2	0	0	0.043s	100
ComentarioDAOJdbcImplTest	com.sv.grupo10.asociacioncomunal.backendasociacioncomunal.dao.impl.ComentarioDAOJdbcImplTest	5	0	0	3.496s	100
ComunicadoDAOJdbcImplTest	com.sv.grupo10.asociacioncomunal.backendasociacioncomunal.dao.impl.ComunicadoDAOJdbcImplTest	2	0	0	0.017s	100
CuotaDAOJdbcImplTest	com.sv.grupo10.asociacioncomunal.backendasociacioncomunal.dao.impl.CuotaDAOJdbcImplTest	2	0	0	0.024s	100
EmailLogDAOJdbcImplTest	com.sv.grupo10.asociacioncomunal.backendasociacioncomunal.dao.impl.EmailLogDAOJdbcImplTest	2	0	0	0.020s	100
MovimientoDAOJdbcImplTest	com.sv.grupo10.asociacioncomunal.backendasociacioncomunal.dao.impl.MovimientoDAOJdbcImplTest	2	0	0	0.020s	100
ProyectoDAOJdbcImplTest	com.sv.grupo10.asociacioncomunal.backendasociacioncomunal.dao.impl.ProyectoDAOJdbcImplTest	5	0	0	0.035s	100
SolicitudDAOJdbcImplTest	com.sv.grupo10.asociacioncomunal.backendasociacioncomunal.dao.impl.SolicitudDAOJdbcImplTest	2	0	0	0.020s	100
VotoDAOJdbcImplTest	com.sv.grupo10.asociacioncomunal.backendasociacioncomunal.dao.impl.VotoDAOJdbcImplTest	3	0	0	0.024s	100
PlantillaEmailTest	com.sv.grupo10.asociacioncomunal.backendasociacioncomunal.models.plantillas.PlantillaEmailTest	1	0	0	0.003s	100
RolesTest	com.sv.grupo10.asociacioncomunal.backendasociacioncomunal.security.RolesTest	4	0	0	0.002s	100
SecretariaServiceImplTest	com.sv.grupo10.asociacioncomunal.backendasociacioncomunal.services.impl.SecretariaServiceImplTest	3	0	0	0.130s	100
TesoreriaServiceImplTest	com.sv.grupo10.asociacioncomunal.backendasociacioncomunal.services.impl.TesoreriaServiceImplTest	6	0	0	0.059s	100
UsuarioServiceImplTest	com.sv.grupo10.asociacioncomunal.backendasociacioncomunal.services.impl.UsuarioServiceImplTest	12	0	0	0.701s	100
NotificacionCorreoTest	com.sv.grupo10.asociacioncomunal.backendasociacioncomunal.services.NotificacionCorreoTest	2	0	0	0.057s	100

Generated by Gradle 9.7.1 at 25 oct. 2026 13:16:48

Figura 9. Reporte de Gradle con las 57 pruebas exitosas

Limitación conocida. Los DAO piden la conexión directamente al `DataSource`, por lo que el `@Transactional` de las pruebas de integración no revierte sus inserciones: cada ejecución deja filas de prueba en la base local. Por eso se recomienda correr la suite sobre una base de prueba, o volver a sembrar la base de la demo después de ejecutarla.

Git y GitHub

El código está en un repositorio público de GitHub: https://github.com/Marcos6192005/AsociacionComunal-Grupo10. El frontend y el backend conviven en el mismo repositorio. Cada cambio se trabajó en una rama propia y se integró a `main` con un Pull Request que describe el cambio y su plan de pruebas. En la entrega final las ramas se apilaron por dependencia (base de datos, luego cada módulo, luego excepciones, concurrencia, pruebas y frontend) y se integraron en ese orden.

Tabla 16. Pull Requests de la entrega final

PR	Rama	Cambio	Autor
#27	feature/jdbc-datasource	Docker, esquema, `DataSourceConfig` y usuarios con JDBC	Ezzz-Lui
#28	tests/usuarios-roles	Pruebas de `Roles` y `UsuarioServiceImpl`	Ezzz-Lui
#29	feature/proyectos-jdbc	Proyectos y comentarios con JDBC	Anderson-Flores-P
#30	fix/base-config	Ajustes de configuración base y presidente único	Ezzz-Lui y Erazo93
#31 a #37	feature/jdbc-*	Votos, solicitudes, actas, comunicados, cuotas, movimientos y bitácora con JDBC	Ezzz-Lui
#38	feature/custom-exceptions	Excepciones propias y `@RestControllerAdvice`	Ezzz-Lui
#39	feature/concurrent-cuotas	Generación concurrente de cuotas	Ezzz-Lui
#40	test/unit-coverage	Pruebas de services, plantillas y controller	Ezzz-Lui
#41	feature/frontend-feedback	Mensajes de error, botón de cuotas y confirmaciones	Ezzz-Lui
Esta entrega	docs/avance3	Documento final, README y corrección de una prueba	Equipo

Los mensajes de commit siguen la convención que pide la guía: `feat:` para funcionalidades, `fix:` para correcciones, `test:` para pruebas, `refactor:` para reorganizaciones y `docs:` para documentación. Ejemplos reales: `feat: persist cuotas with JDBC`, `feat: generate monthly fees on a thread pool`, `test: cover services, templates and treasury HTTP errors`.

Usuarios de GitHub de los integrantes: Luis Gerardo Ramos Martínez aparece como Ezzz-Lui y Luis Ramos; Marcos Armando Vásquez González como Marcos6192005 (dueño del repositorio); Anderson Steven Flores Pérez como Anderson-Flores-P y Anderson Flores; y Roberto De Jesus Erazo Ramirez como Erazo93.

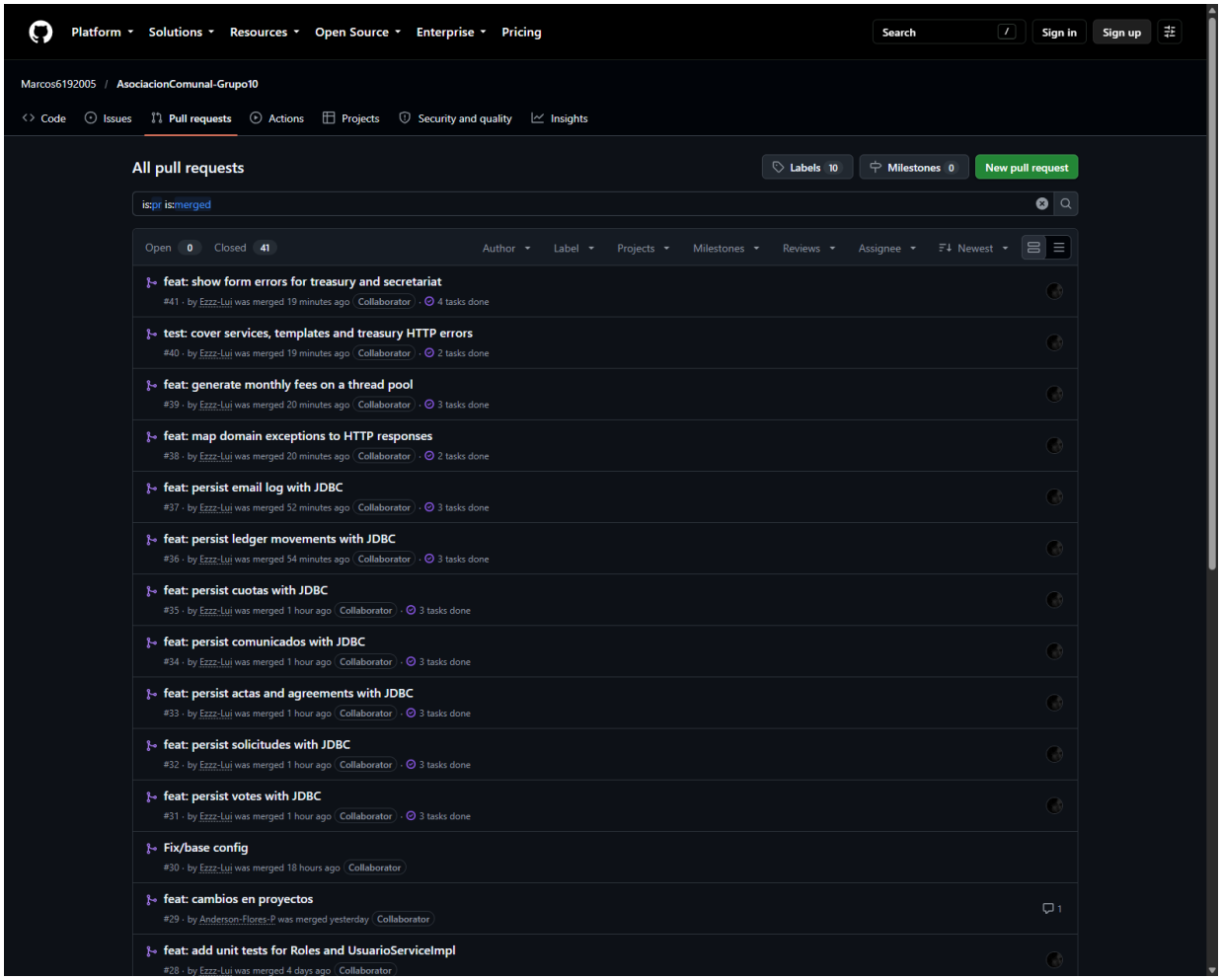


Figura 10. Pull Requests integrados en GitHub

Conclusiones

La entrega final cierra la evolución del proyecto: del diseño del avance 1, al sistema funcional del avance 2, a una aplicación que persiste en PostgreSQL, usa un pool de conexiones justificado y ejecuta trabajo concurrente real contra la base. La decisión del avance 2 de programar los services contra interfaces DAO fue la que hizo posible migrar diez DAO a JDBC sin reescribir la lógica de negocio.

La concurrencia tiene una finalidad concreta: generar las cuotas del mes en paralelo, con una conexión por hilo, y dejar que la restricción `UNIQUE` de la base resuelva el caso de dos ejecuciones simultáneas. Llevar esa validación a la base fue más seguro que confiar solo en una verificación en Java, que no protege contra dos hilos que consultan al mismo tiempo.

Las excepciones propias y el manejador global hicieron más legibles los controllers y dieron al usuario mensajes claros. Las 57 pruebas permiten cambiar código con confianza, y el ejercicio de escribirlas expuso un problema real: las pruebas de integración escriben en la base de desarrollo, lo que queda documentado como mejora pendiente junto con mover la contraseña del correo a una variable de entorno.

Por último, la defensa es individual: los cuatro integrantes revisaron el pool, la concurrencia, las excepciones y las pruebas, no solo la parte que desarrollaron.

Bibliografía

- Oracle. (2024). The Java Tutorials: JDBC Database Access y Concurrency. https://docs.oracle.com/javase/tutorial/
- Apache Software Foundation. (2024). The Tomcat JDBC Connection Pool. https://tomcat.apache.org/tomcat-10.1-doc/jdbc-pool.html
- PostgreSQL Global Development Group. (2024). PostgreSQL 16 Documentation: Error Codes (SQLSTATE 23505). https://www.postgresql.org/docs/16/errcodes-appendix.html
- Broadcom. (2024). Spring Boot Reference Documentation. https://docs.spring.io/spring-boot/documentation.html
- JUnit Team. (2024). JUnit 5 User Guide. https://junit.org/junit5/docs/current/user-guide/
- Mockito. (2024). Mockito Documentation. https://site.mockito.org/
- Vercel. (2024). Next.js Documentation: App Router y Server Actions. https://nextjs.org/docs
- Schwaber, K. y Sutherland, J. (2020). La Guía de Scrum. Scrum.org. https://scrumguides.org/
- Guevara Gómez, D. E. (2026). Entrega final proyecto. Programación II. Universidad Evangélica de El Salvador.